Software Engineering

Typescript Features and Paradigms 2

What we have learned...

- Advanced Type System
- Async Programming with TypeScript
- Interface
- Intersection and Type

What we will learn...

- Module Resolution and System
- Enums
- TypeScript Best Practices

Get to know our topic!

Module Resolution

Module Resolution and System in TypeScript

Module resolution is the process TypeScript uses to find and load external modules. It is crucial for organizing and managing dependencies.

```
// Importing a module
import { someFunction } from './myModule';
```

Module Resolution and System in TypeScript

In a Node.js project, you might have a **file structure** with different **modules** for **routes**, **controllers**, and **models**. Module resolution helps in connecting these modules.

Think of module resolution like **finding a book** in a **library**. You specify the book's location (path) to get the information you need.

Module resolution ensures that TypeScript can find and link the necessary modules during the compilation process, facilitating organization and scalability.

Create 2 files.

- 1st file cart.ts : make a function that can print the latest cart items, where the function accept an array of string.
- 2nd file index.ts : import that function, and call it.

Now convert the export function to be come a module object instead

Enum

Enum

An **enum** (enumeration) in TypeScript is a way to define a set of named constant values, representing a distinct set of possibilities.

Enums make it easier to work with a **set** of **related values** and improve code readability.

Enum

Think of an enum as a menu with options. If you have a menu for a coffee shop, the options could be "**Espresso**," "**Latte**," "**Cappuccino**," and "**Mocha**."

Each option represents a **specific choice**, similar to how enum values represent distinct options in code.

Enums provide a way to create named sets of constants, making the code more readable and self-explanatory. They help avoid using "**magic numbers**" and improve code maintainability.

In a game development project, you might use enums to represent different **directions** a character can **move**, making the code more expressive.

Enum

```
enum Direction{
 up =1,
 down,
 left,
 right
}

let move: Direction = Direction.up;
console.log(move); // Output: 1
```

```typescript
enum Status {
  Pending,
  InProgress,
  Completed,
}

function getStatusMessage(status: Status): string {
  switch (status) {
    case Status.Pending:
      return "The task is pending.";
    case Status.InProgress:
      return "The task is in progress.";
    case Status.Completed:
      return "The task is completed.";
    default:
      return "Unknown status.";
  }
}

console.log(getStatusMessage(Status.InProgress));
// Output: The task is in progress.
```

Enum in Error

```
enum LogLevel {
  Error = "ERROR",
  Warn = "WARN",
  Info = "INFO",
  Debug = "DEBUG",
}
```

Make a function to set the transaction state to be completed, use enum for transaction state.

Clue :

- Define a variable of a pending transaction.
- Make a function to set the variable transaction, change the state to be completed.

Parameter 1 : transaction object

Parameter 2: desired transaction state

TypeScript Best Practices

Following best practices in TypeScript ensures clean, maintainable, and efficient code.

1. Use TypeScript's Strict Mode

Why ?

Enabling strict mode ("strict": true in tsconfig.json) helps catch common errors during development, making the code more reliable and maintainable.

How ?

Include "strict": true in your tsconfig.json file.

1. Use TypeScript's Strict Mode

```
// tsconfig.json
{
  "compilerOptions": {
    "strict": true
  }
}
```

```
// Without Strict Mode
let myVar1: string = null; // No error

// With Strict Mode
let myVar2: string = null; // Error: Type 'null' is not assignable
```

- **Without Strict Mode:**

TypeScript does not enforce strict type-checking.
Assigning null to a variable declared as string is allowed, as null is treated as a valid value for all types.

- **With Strict Mode:**

Strict mode enables stricter type-checking features, such as strictNullChecks.
With strictNullChecks enabled, null is not considered a valid value for the type string unless explicitly included (e.g., string | null).

```typescript
// Without Strict Mode
function add(x, y) {
  return x + y;
}

// With Strict Mode
function addStrict(x: number, y: number): number {
  return x + y;
}
```

- **Without Strict Mode:**

The parameters x and y are not explicitly typed, which means they can be of any type. This flexibility allows errors to occur at runtime if x or y are non-numeric types.

- **With Strict Mode:**

TypeScript enforces type annotations for function parameters and return values when strict is enabled. The addStrict function specifies that both x and y must be numbers (x: number, y: number). It also explicitly declares the return type as a number (: number).

```typescript
// Without Strict Mode
interface Person {
  name: string;
}

let person: Person = {
  name: "John",
  age: 25, // No error, but 'age' is not part of the 'Person' interface
};

// With Strict Mode
interface StrictPerson {
  name: string;
}

let strictPerson: StrictPerson = {
  name: "John",
  age: 25, // Error: Object literal may only specify known properties,
```

2. Type Every Variable and Function

Why ?

Explicitly typing variables and functions improves code readability, provides better tooling support, and catches potential bugs early.

How ?

Always add types to variables and function parameters.

2. Type Every Variable and Function

```
// Explicit typing for variables
let count: number = 5;

// Explicit typing for function parameters and return type
function addNumbers(a: number, b: number): number {
  return a + b;
}
```

3. Use Interfaces or Types for Data Structures

Why ?

Interfaces and types provide a clear and reusable way to define data structures, enhancing code maintainability.

How ?

Define interfaces or types for complex data structures.

3. Use Interfaces or Types for Data Structures

```typescript
// Using an interface for a user object
interface User {
  id: number;
  username: string;
  email: string;
}

// Function accepting a User object
function printUser(user: User): void {
  console.log(`User: ${user.username} (${user.email})`);
}
```

4. Avoid Using the any Type:

Why ?

Using any weakens TypeScript's static typing, defeating its purpose. It can lead to runtime errors that could have been caught during development.

How ?

Avoid using any whenever possible; instead, specify the correct types.

4. Avoid Using the any Type

```
// Avoid using any
function unpredictableFunction(data: any): void {
  // Code that might lead to runtime errors
}
```

5. Use Union Types and Enums for Clearer Code:

Why ?

Union types and enums make the code more expressive and self-documenting.

How ?

Use union types for variables that can have multiple types and enums for a set of related values.

5. Use Union Types and Enums for Clearer Code:

```
// Using a union type
type Result = string | number;

// Using an enum
enum Status {
  Pending,
  Approved,
  Rejected
}

// Function with union type parameter
function processResult(result: Result): void {
  // Code here
}

// Function with enum parameter
function updateStatus(status: Status): void {
  // Code here
}
```

6. Use Modules to Organize Code

Why ?

Modules help organize and encapsulate code, preventing naming collisions and improving code structure.

How ?

Split your code into modules, each containing related functionality.

6. Use Modules to Organize Code

```
// File: mathOperations.ts
export function add(a: number, b: number): number {
  return a + b;
}

// File: geometry.ts
export function calculateCircleArea(radius: number): number {
  return Math.PI * Math.pow(radius, 2);
}

// File: main.ts
import { add } from './mathOperations';
import { calculateCircleArea } from './geometry';

const sum = add(3, 5);
const circleArea = calculateCircleArea(10);
```

7. Use Async/Await for Asynchronous Operations

Why ?

Async/await syntax provides a more readable and maintainable way to work with asynchronous code compared to callbacks or promises.

How ?

Use async functions and await for asynchronous operations.

7. Use Async/Await for Asynchronous Operations

```
// Using async/await with a Promise
async function fetchData(): Promise<string> {
  const response = await fetch('https://api.example.com/data');
  const data = await response.text();
  return data;
}
```

7. Use Async/Await for Asynchronous Operations

The **async/await** syntax is used to handle the **asynchronous** nature of the API fetch operation. The **await** keyword is used to pause the execution of the function until the Promise returned by **fetch()** and **response.text()** are resolved.

By using async/await, the asynchronous logic is **expressed** in a more **synchronous-looking way**, making the code more intuitive to understand.

8. Use Descriptive Variable and Function Names

Why ?

Descriptive names improve code readability and maintainability, making it easier for developers to understand the purpose of variables and functions.

How ?

Choose meaningful names that reflect the purpose of your variables and functions.

8. Use Descriptive Variable and Function Names

```
// Poorly named variables
let x = 10;
let y = 20;

// Well-named variables
let totalPrice = 10;
let discountAmount = 20;
```

9. Follow Code Formatting and Style Guides

Why ?

Consistent code formatting and style improve collaboration among team members and maintainability.

How ?

Adopt a code formatting and style guide, and use tools like prettier or tslint to enforce it.

9. Follow Code Formatting and Style Guides

```
// Inconsistent formatting
function   poorlyFormattedFunction  (value:number):number{
    return value + 1;
}

// Consistent formatting
function wellFormattedFunction(value: number): number {
  return value + 1;
}
```

10. Mistakes and what's better

```
// Mistake
function add(a, b) {
  return a + b;
}

// Better
function add(a: number, b: number): number {
  return a + b;
}
```

10. Mistakes and what's better

```
// Mistake
function dynamicFunction(value: any) {
  // ...
}

// Better
function dynamicFunction(value: unknown) {
  if (typeof value === "string") {
    // Handle string case
  }
}
```

10. Mistakes and what’s better

```
// Good
function calculateTotal(price: number, quantity: number): number {
  return price * quantity;
}
```

Let’s play!
Fix the code!

Variable

```
let itemName = "laptop"
```

Module Resolution

```
import someFunction require "./someModule"
```

Variable

```
let myProfile : {
  username: string,
  email: string
} = {
  username: 'anonymous',
  email: 'anon@email.com'
}
```

Type Annotation

```
function transact(balance: number, amount) {
  return balance - amount;
}

transact(10000, 8000);
```

Type Annotation

```
function printExitMessage() {
  console.log("Press backagain to exit app");
}

printExitMessage();
```

Rest Parameter

```
function printIds( ... ids, userId) {
  console.log("Press back again to exit app");
}

printIds(1,2,3,4, "user with id 99");
```

Variable

```
let assgnmnts = ['Assigment 1', 'Assigment 2', 'Assigment 3'];
```

Spread Operator

```
let assgnmnts = "'Assigment 1', 'Assigment 2', 'Assigment 3'";
getUserProfile(student, ...assignments);
```

Type Intersection

```
type acceptedPayment = Bank | Ewallet;
```

Enum

```
enum Status {
  Grow,
  Confess,
  Accepted,
  Rejected,
}
```

Module Resolution

```
function export printHello() {
  console.log('Hello!')
}
```

Module Resolution

```
export module someFunction;
```

QnA & Discussion

Today’s Recap

External Resources

- 📖 **Module Resolution and System:**
  - TypeScript Module Resolution
  - Understanding TypeScript's module resolutions
- 📖 **Using External Libraries with TypeScript:**
  - Using JavaScript Libraries in TypeScript Projects
  - DefinitelyTyped Repository
- 📖 **TypeScript and Frontend Frameworks:**
  - TypeScript and React
  - Angular and TypeScript
- 📖 **Error Handling and Debugging in TypeScript:**
  - TypeScript - Debugging
  - Error Handling in TypeScript
- 📖 **TypeScript Best Practices:**
  - TypeScript Deep Dive - Best Practices
  - Airbnb TypeScript Style Guide.

Thank you!

Trust is an unspoken rules,
Breaking rules, you have another chance,
But breaking trust, you will not have another one.

Share your learning journey with us!
Let's inspire each other with your stories

@revou_id RevoU revoudotco